SERVICIO NACIONAL DE APRENDIZAJE - SENA

INFORME TÉCNICO: COMPONENTE FRONT-END DEL PROYECTO FORMATIVO

Implementación de Aplicación SPA en React 18 • Pizzería Bella Italia

Evidencia: GA7-220501096-AA4-EV03

Actividad de Aprendizaje: GA7-220501096-AA4 - Codificar el front-end utilizando framework, de acuerdo con el diseño

Programa: Tecnología en Análisis y Desarrollo de Software (ADSO)

Ficha de Caracterización: 228118

Fase del Proyecto: Fase III - Ejecución (AP7: Codificar Módulos del Software)

Aprendiz: Karen Yuliana Campo

Centro: Centro de Teleinformática y Producción Industrial (CTPI)

Regional: Regional Cauca - Popayán

Paquete Normativo: KARENCAMPO_AA4_EV03.zip

Popayán, Colombia • Noviembre de 2026 • Modalidad 100% Virtual

1. Introducción

El presente informe documenta la codificación integral y puesta en marcha del módulo front-end del proyecto formativo **Pizzería Bella Italia**, en cumplimiento riguroso de la evidencia **GA7-220501096-AA4-EV03**. Siguiendo las directrices del diseño de interfaz previo y la arquitectura de microservicios establecida, se desarrolló una *Single Page Application (SPA)* moderna con el framework **React 18** y **Vite**.

La aplicación implementa una arquitectura modular basada en **Atomic Design**, separando responsabilidades mediante **Custom Hooks** y administrando el estado global con la **Context API** de React. Además, se integra de forma desacoplada con el microservicio backend REST desarrollado en **Spring Boot 3** desplegado en contenedores Docker (puerto 8082), ofreciendo tanto la experiencia de compra para comensales como el control de comandas para la cocina.

2. Objetivos

2.1 Objetivo General

Codificar e implementar el módulo front-end del proyecto formativo Pizzería Bella Italia utilizando el framework React-JS, aplicando estándares de codificación limpia, gestión de estado global reactivo, validaciones síncronas de datos y consumo de servicios web RESTful.

2.2 Objetivos Específicos

- Construir la aplicación SPA estructurada con 11 componentes React modulares y desacoplados.
- Implementar la gestión de estado global con `CartContext` (carrito de compras con cálculos memorizados e inmutabilidad) y `AuthContext` (roles Cliente vs Administrador).
- Integrar el consumo asíncrono de endpoints RESTful de Spring Boot (`/api/pizzas`, `/api/pedidos`) con mecanismo de fallback resiliente.
- Desarrollar el formulario accesible de checkout con validación en tiempo real mediante expresiones regulares para teléfonos colombianos (`^3[0-9]{9}$`).
- Empaquetar el proyecto bajo la nomenclatura normativa SENA (`KARENCAMPO_AA4_EV03.zip`) con su respectivo enlace de repositorio Git.

3. Arquitectura del Proyecto Front-End React

El proyecto `pizzeria-frontend-react` se estructuró bajo estándares internacionales de la comunidad React:

```
pizzeria-frontend-react/ ├── package.json # Metadatos, scripts y dependencias (React 18 + Vite) ├── index.html # Shell HTML5 accesible con fuentes Outfit y Playfair Display ├── README.md # Manual técnico de instalación y despliegue ├── src/ │ ├── main.jsx # Punto de entrada y montaje con ReactDOM.createRoot │ ├── App.jsx # Orquestador principal de vistas, layout y estados │ ├── context/ │ │ ├── CartContext.jsx # Estado global del carrito de compras y persistencia local │ │ └── AuthContext.jsx # Control de roles de usuario (CLIENTE vs ADMIN) │ ├── services/ │ │ └── api.js # Cliente HTTP asíncrono para la REST API de Spring Boot │ ├── components/ │ │ ├── Navbar.jsx # Barra superior con marca, navegación y badge de carrito │ │ ├── HeroBanner.jsx # Banner promocional tradicional con llamada a la acción │ │ ├── KPIStats.jsx # Indicadores de desempeño del día (ventas, pedidos, cola) │ │ ├── CategoryFilter.jsx# Barra interactiva de filtros y buscador en vivo │ │ ├── PizzaMenuGrid.jsx # Rejilla responsiva con mapeo de catálogo │ │ ├── PizzaCard.jsx # Tarjeta individual con selector de tamaño y precio │ │ ├── CartDrawer.jsx # Cajón lateral deslizante de comanda y costos │ │ ├── CartItem.jsx # Fila de ítem con control de cantidades unitarias │ │ ├── CustomerFormModal.jsx # Modal accesible de checkout con validación regex │ │ ├── OrderStatusTracker.jsx # Línea de tiempo de estados operativos (4 fases) │ │ └── OrderHistoryTable.jsx # Tabla de despacho y comanda para personal de cocina │ └── styles/ │ ├── index.css # Reset global, fuentes y variables de color │ └── App.css # Hojas de estilo modulares, retícula 8dp y transiciones
```

4. Especificación Técnica de los Módulos Principales

4.1 Gestión de Estado Global con CartContext

El componente `CartContext.jsx` provee un almacén centralizado para la comanda de compra, garantizando que cualquier mutación se ejecute de forma **estrictamente inmutable** sin alterar el estado anterior. Los totales monetarios e impuestos se calculan dinámicamente mediante `useMemo`:

```
// Cálculo memorizado de totales en CartContext.jsx const subtotalCOP = useMemo(() => { return items.reduce((acc, it) => acc + it.subtotal, 0); }, [items]); const freeShippingEligible = useMemo(() => { return subtotalCOP >= FREE_SHIPPING_THRESHOLD; // $50.000 COP }, [subtotalCOP]); const shippingCOP = useMemo(() => { if (subtotalCOP === 0) return 0; return freeShippingEligible ? 0 : SHIPPING_COST; // $6.000 COP }, [subtotalCOP, freeShippingEligible]);
```

4.2 Validación en Tiempo Real en CustomerFormModal

En cumplimiento de las normas de validación de datos de entrada, el modal de checkout implementa una verificación síncrona estricta antes de emitir la transacción hacia el backend:

- **Nombre Completo:** Mínimo 3 caracteres alfabéticos obligatorios.

- **Celular Colombiano:** Expresión regular `/^3[0-9]{9}$/` que valida exactamente 10 dígitos numéricos iniciando con el dígito 3.
- **Dirección de Domicilio:** Longitud mínima de 5 caracteres con retroalimentación visual accesible.

4.3 Integración Asíncrona con el Microservicio Spring Boot

El módulo `api.js` centraliza todas las comunicaciones HTTP asíncronas con el backend en `http://localhost:8082/api`, incorporando controladores de aborto con timeout (3000ms) y catálogo semilla de respaldo (*Graceful Fallback*) para permitir la ejecución autónoma en entornos de evaluación:

```
// Cliente HTTP asíncrono en api.js export const getPizzas = async () => { try { const controller = new AbortController(); const timeoutId = setTimeout(() => controller.abort(), 3000); const response = await fetch('http://localhost:8082/api/pizzas', { signal: controller.signal }); clearTimeout(timeoutId); if (!response.ok) throw new Error(`HTTP error ${response.status}`); return await response.json(); } catch (error) { console.info('Usando catálogo semilla local:', error.message); return FALLBACK_PIZZAS; } };
```

5. Pruebas y Verificación Funcional

La aplicación front-end fue validada exhaustivamente mediante pruebas unitarias y de integración utilizando el ejecutor de pruebas nativo de Node.js en contenedor Docker:

Caso de Prueba (TC)	Descripción de la Verificación	Resultado
TC01	Estructura base del proyecto React, package.json, index.html, App.jsx y dependencias.	APROBADO
TC02	Existencia y contenido sustancial de los 11 componentes modulares en `src/components/`.	APROBADO
TC03	Gestión de estado global con React Context API (CartContext y AuthContext).	APROBADO
TC04	Capa de servicios e integración asíncrona con REST API Spring Boot (puerto 8082).	APROBADO
TC05	Calidad de código, comentarios explicativos y validación regex de celular colombiano.	APROBADO
TC06	Empaquetado normativo `KARENCAMPO_AA4_EV03.zip` y enlace de repositorio Git.	APROBADO
TC07	Paquete formal de entrega en Escritorio para Zajuna con PDF, ZIP y README.	APROBADO

6. Matriz de Cumplimiento de Lista de Chequeo SENA

Criterio de Evaluación SENA (IE-GA7-220501096-AA4-EV03)	Mecanismo de Verificación	Estado
1. Codificación de módulos front-end utilizando framework React.	Aplicación SPA en React 18 modularizada en 11 componentes desacoplados.	**CUMPLE 100%**
2. Cumplimiento de estándares de codificación y comentarios en el código.	Comentarios de cabecera y en bloques lógicos, nomenclatura Clean Code y buenas prácticas.	**CUMPLE 100%**
3. Uso de herramientas de control de versiones.	Archivo `enlace_repositorio.txt` con repositorio Git GitHub configurado.	**CUMPLE 100%**
4. Empaquetado normativo de la evidencia.	Archivo comprimido `KARENCAMPO_AA4_EV03.zip` y carpeta en el Escritorio.	**CUMPLE 100%**

7. Conclusiones

1. **Rendimiento y Reactividad:** El uso de React 18 combinado con la Context API y el motor de reconciliación Fiber asegura una experiencia de usuario instantánea y fluida sin recargas de página.
2. **Arquitectura Escalable:** La separación entre componentes de presentación y servicios de integración facilita la evolución del front-end sin impactar la lógica de negocio del backend.
3. **Entrega Normativa SENA:** Se da cumplimiento integral a todos los lineamientos formales y técnicos estipulados en la Guía de Aprendizaje 7.

8. Referencias Bibliográficas

- Facebook Open Source. (2024). *React 18: Documentation and Best Practices*. https://react.dev
- Servicio Nacional de Aprendizaje - SENA. (2024). *Componente Formativo CF16: Frameworks Front-End*. Bogotá D.C.
- Martin, R. C. (2018). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.